

day 32 | 251112 W

hitherto

We started today by finishing up something that we were working on from last time, namely

1. how to get jupyter to automagically import the statements that we typically put at the front
2. how to get our own library of scripts that we have commonly used so that we can import them

The students who were on windows had received much less guidance on how to get either of these to work, particularly the first one. So we worked for some time in getting this to work for both groups of students. I have up to date notes on how to accomplish both of these items in future semesters in the day 31 notes, so check those out in the future.

herein

We began by talking through what a partial differential equation actually is. To understand this, you first need to understand what a partial derivative is:

$$f(x, y, z) = 4xy^2z^4$$

$$\frac{\partial f}{\partial x} = 4y^2z^4$$

$$\frac{\partial f}{\partial y} = 8xyz^4$$

So a partial derivative is a derivative of a function with respect to one single variable, treating the other variables as constants. Not too bad right. Furthermore, second partial derivatives are second derivatives treating the other variables as constants, but this can be a little bit trickier:

$$\frac{\partial^2 f}{\partial x^2} = 0$$

$$\frac{\partial^2 f}{\partial x \partial y} = 8yz^4$$

Do you see that? I can take a partial derivative with respect to one variable on a function that is already a partial derivative of a different variable. Tricky stuff and you

have to be careful. BUT!

We are not solving partial derivatives using analytical techniques, but using **numerical techniques**. Which actually eases the burden quite a bit. However each partial differential equation, will need to be taken one at a time, in order to come up with a method to numerically solve it. Fortunately, most second order partial differential equations in physics take on just a few forms, so we should be able to learn a couple of techniques, and then apply them to many equations.

So first, we will take **Laplace's Equation**, and solve it in 2 dimensions. Laplace's equation looks like this:

$$\nabla^2 V = 0$$

Now that upside down *delta* character looks bad. But another way to write this equation is like this:

$$\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = 0$$

So, to solve such an equation we also need **boundary conditions**. These will take the form of what is the potential around the outside of this region? For a square/rectangleish region this might be something like the potential (V) is zero along all of the edges except the top boundary (have a look at p 407). So, in order to solve this, we will use the relaxation technique, where the solution at any particular point is the average of the solution at all of the points all around it. You may be seeing the circularity of the kind of calculation we are about to do. The value of the potential at any point depends on the value at every other point, which depends on the value at the other points and so on...

So to pull this off, first we are going to use excel to do this. You have to enable a setting that allows recursive calculations. But once you have done this you can run that method over and over (Ctrl+Shift+F9 on my computer/program) and this will eventually give you the answer that you can plot. Plotting this in excel is horrible, so I recommend plotting it in jupyter, and I show you below how to do that.

hence

This did not all come together in one class, because we spent too much time going over the library importing business mentioned in hitherto. Hopefully by next time, I will have sped students up by giving them the correct instructions on doing that stuff before hand. C'est la vie.

```
In [1]: import sys
        sys.path
```

```
Out[1]: ['/home/ehreming/Dropbox/390-2025-fall/scripts',  
        '/home/ehreming/anaconda3/lib/python312.zip',  
        '/home/ehreming/anaconda3/lib/python3.12',  
        '/home/ehreming/anaconda3/lib/python3.12/lib-dynload',  
        '',  
        '/home/ehreming/anaconda3/lib/python3.12/site-packages']
```

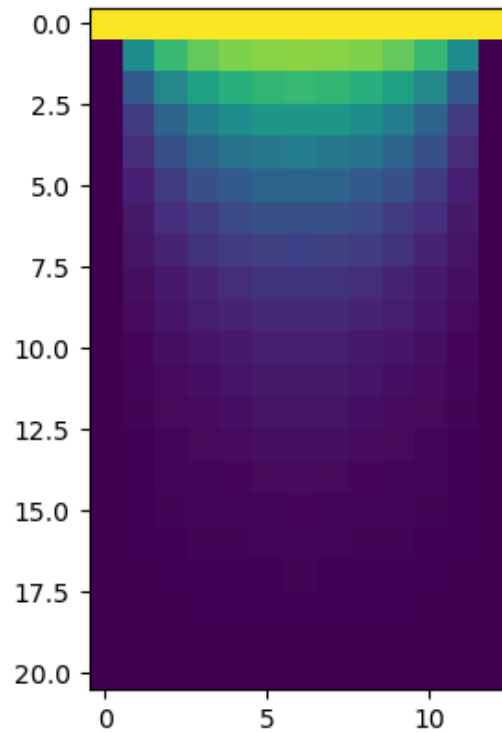
```
In [7]: data = pd.read_excel('../data/laplace.xlsx', header=None)
```

```
In [8]: f = data.to_numpy()
```

```
In [9]: fig0, ax0 = plt.subplots()  
ax0.imshow(f)
```

```
Out[9]: <matplotlib.image.AxesImage at 0x7967765cd010>
```

Figure



```
In [ ]:
```